

Matrix Sum Report

1. Why is the program so slow?

Original code

```
GNU nano /2
#include <iostream>
#include <ctime>
#include <chrono>

#define N 10000000
#define M (64 / sizeof(int)) // Cache line size: `getconf LEVEL1_DCACHE_LINESIZE`

int mat[N][M];

int main() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            mat[i][j] = 1;
        }
    }

    int sum = 0;

    const auto t1 = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            sum += mat[j][i];
        }
    }
    const auto t2 = std::chrono::high_resolution_clock::now();
    const auto int_ms = std::chrono::duration_cast<std::chrono::milliseconds>(t2 - t1);

    std::cout << "took: " << int_ms.count() << " ms" << std::endl;
    std::cout << "sum: " << sum << std::endl;
}
```

The program is slow because its memory access pattern has poor spatial locality. A cache line is typically 64 bytes and arrays benefit from spatial locality. If the program accesses matrix elements in an order that does not reuse nearby data well, cache misses increase, and each cache miss can stall the CPU. Therefore, changing the traversal order to access nearby elements consecutively improves cache utilization and performance.

The sum loop does:

```
for (int i = 0; i < M; i++) {
    for (int j = 0; j < N; j++) {
        sum += mat[j][i];
    }
}
```

```
}
```

So the access order is:

mat[0][0], mat[1][0], mat[2][0], ...

This is **column-wise access** on a **row-major array**.

Step by step:

1. CPU loads the cache line containing mat[0][0].
2. That cache line actually contains the whole row: mat[0][0] ~ mat[0][15].
3. But the next access is mat[1][0], which is in a **different row**, so a **different cache line**.
4. So most of the previous cache line is wasted.
5. This repeats over and over.

So each time the CPU fetches **64 bytes**, it immediately uses only **4 bytes** and then jumps away. That gives poor cache usage, more cache misses, and more stalls. Every cache miss can cause a **pipeline stall**, and DRAM is much slower than cache .

2. Verify the idea using perf stat -d ./mat_sum

```
student@hpc-i-shared:~/蔡定宇$ ./mat_sum
took: 830 ms
sum: 160000000
student@hpc-i-shared:~/蔡定宇$ perf stat -d ./mat_sum
took: 921 ms
sum: 160000000

Performance counter stats for './mat_sum':

      1890.44 msec task-clock                #   0.964 CPUs utilized
         3      context-switches             #   1.587 /sec
         0      cpu-migrations               #   0.000 /sec
    156376      page-faults                  #  82.719 K/sec
  4519592487      cycles                    #   2.391 GHz           (85.69%)
  311861259      stalled-cycles-frontend    #   6.90% frontend cycles idle   (85.74%)
  4331738192      instructions              #   0.96   insn per cycle          (85.68%)
                                          #  0.07  stalled cycles per insn
   430774146      branches                  #  227.870 M/sec          (85.76%)
   17083817      branch-misses              #   3.97% of all branches   (85.72%)
  1801294724      L1-dcache-loads           #  952.843 M/sec          (85.70%)
  152522772      L1-dcache-load-misses      #   8.47% of all L1-dcache accesses (85.72%)
<not supported>      LLC-loads
<not supported>      LLC-load-misses

   1.960580385 seconds time elapsed

   1.468320000 seconds user
   0.492054000 seconds sys

student@hpc-i-shared:~/蔡定宇$
```

3. How to improve the performance?

Modified code

```
GNU nano 7.2
#include <iostream>
#include <ctime>
#include <chrono>

#define N 10000000
#define M (64 / sizeof(int)) // Cache line size: `getconf LEVEL1_DCACHE_LINESIZE`

int mat[N][M];

int main() {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < M; j++) {
            mat[i][j] = 1;
        }
    }

    int sum = 0;

    const auto t1 = std::chrono::high_resolution_clock::now();
    for (int j = 0; j < N; j++) {
        for (int i = 0; i < M; i++) {
            sum += mat[j][i];
        }
    }
    const auto t2 = std::chrono::high_resolution_clock::now();
    const auto int_ms = std::chrono::duration_cast<std::chrono::milliseconds>(t2 - t1);

    std::cout << "took: " << int_ms.count() << " ms" << std::endl;
    std::cout << "sum: " << sum << std::endl;
}
```

Use **row-wise traversal** in the sum loop.

```
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        sum += mat[i][j];
    }
}
```

Why this is faster

A **cache line** is typically **64 bytes**, and performance improves when code has good **cache locality**, especially **spatial locality** for arrays. Every **cache miss** can cause a **pipeline stall**.

Step by step

Access order is:

mat[0][0], mat[0][1], ..., mat[0][15]

1. CPU loads the cache line containing mat[0][0].

2. That cache line contains the whole row: `mat[0][0] ~ mat[0][15]`.
3. The next access is `mat[0][1]`, so it is still in the same cache line.
4. The CPU keeps using `mat[0][2] ... mat[0][15]` from that same cache line.
5. So one cache-line fetch is fully used.
6. Then CPU loads the next cache line for `mat[1][0] ~ mat[1][15]`, moves to the next row and loads the next cache line.
7. This repeats row by row.
8. Therefore, cache usage is better, cache misses are fewer, and the program runs faster.

4. Verify that the modified program is faster

Modified version results

```
student@hpc-i-shared:~/蔡定宇$ ./mat_sum
took: 368 ms
sum: 160000000
student@hpc-i-shared:~/蔡定宇$ perf stat -d ./mat_sum
took: 402 ms
sum: 160000000

Performance counter stats for './mat_sum':

    1405.68 msec task-clock                #    0.950 CPUs utilized
         1      context-switches          #    0.711 /sec
         0      cpu-migrations            #    0.000 /sec
    156376      page-faults               # 111.246 K/sec
  3300094300    cycles                    #    2.348 GHz                    (85.90%)
   303814785    stalled-cycles-frontend   #    9.21% frontend cycles idle   (85.94%)
  4648490917    instructions              #    1.41  insn per cycle
                                           #    0.07  stalled cycles per insn (85.89%)
   455876837    branches                  # 324.311 M/sec                    (85.91%)
   16926123     branch-misses             #    3.71% of all branches        (86.01%)
  1826919866    L1-dcache-loads           #    1.300 G/sec                    (85.99%)
   28080906     L1-dcache-load-misses     #    1.54% of all L1-dcache accesses (84.36%)
<not supported> LLC-loads
<not supported> LLC-load-misses

    1.479707191 seconds time elapsed

    1.006103000 seconds user
    0.473704000 seconds sys

student@hpc-i-shared:~/蔡定宇$
```

Time and perf comparison

Runtime comparison

- Original version: 830 ms

- Modified version: 368 ms
- Speedup
 - $830 / 368 \approx 2.26$
 - So the modified version is about $2.26\times$ faster

Perf comparison

- Original
 - L1-dcache-load-misses: 152,522,772
 - L1 miss rate: 8.47%
- Modified
 - L1-dcache-load-misses: 28,080,906
 - L1 miss rate: 1.54%